

Práctica 1: Python básico. Medición del tiempo de ejecución. TAD Conjunto Disjunto y Componentes Conexas

Algoritmia y Estructuras de Datos Avanzadas 2026-27

Nombre del Estudiante: **[Escribe tu nombre aquí]**

Grupo: **[126 o 127]**

8 de octubre de 2026

1 I.A.4 Medición del rendimiento: `has_sum_pair`

Gráfica de tiempos de ejecución

[Inserta aquí tu gráfica comparando el "caso con coincidencia" y el "caso sin coincidencia" para n entre 10 y 10.000]

Análisis y Justificación

¿Cómo varía el tiempo de ejecución en ambos casos a medida que crece n ?

[Escribe tu respuesta aquí. Analiza la tendencia de las curvas observadas en la gráfica para el caso promedio (early exit) y el caso peor (búsqueda fallida).]

¿Mantiene la función una complejidad temporal de $O(n)$ en ambos escenarios?

[Escribe tu justificación aquí. Relaciona los tiempos medidos empíricamente con la complejidad teórica esperada utilizando diccionarios/sets.]

2 I.B.3 Análisis empírico: Algoritmos de codificación RLE

Gráfica de tiempos de ejecución

[Inserta aquí tu gráfica comparando rle_encode_naive frente a rle_encode_optimized]

Análisis y Justificación

¿Cuál es el comportamiento temporal de ambas funciones al hacer crecer n?

[Escribe tu respuesta aquí. Compara las pendientes y el crecimiento (lineal vs cuadrático) de ambas funciones.]

¿Qué impacto tienen las operaciones internas de creación y modificación de listas en Python?

[Escribe tu respuesta aquí. Explica la diferencia teórica y empírica entre concatenar listas creando objetos nuevos con el operador "+" (naive) vs usar el método de modificación in-place "append()" (optimized).]

3 II.C Cuestiones sobre Conjuntos Disjuntos y Componentes Conexas

1. Sin darnos cuenta, en nuestro algoritmo de encontrar CCs podemos pasar listas con ramas repetidas o donde los vértices coinciden con los de una que ya está aunque en orden inverso. ¿Afectará esto al resultado del algoritmo? ¿Por qué?

[Escribe tu respuesta aquí. Razona sobre cómo se comporta la operación Union cuando dos elementos ya pertenecen al mismo representante / Conjunto Disjunto.]

2. Argumentar que nuestro algoritmo de encontrar componentes conexas es correcto, esto es, que a su final en los distintos subconjuntos disjuntos se encuentran los vértices de las distintas componentes del grafo dado.

[Escribe tu respuesta aquí. Puedes argumentar basándote en la propiedad transitiva de la conectividad en grafos y cómo la estructura de Conjuntos Disjuntos agrupa exhaustivamente los vértices de una misma componente.]

3. El tamaño de un grafo no dirigido viene determinado por el número n de nodos y la longitud de la lista l de ramas. Estimar razonadamente en función de ambos el coste del algoritmo de encontrar las componentes conexas mediante conjuntos disjuntos.

[Escribe tu respuesta aquí. Analiza el coste de la inicialización y el coste de iterar sobre la lista de tamaño $|l|$, teniendo en cuenta la complejidad casi constante de las operaciones Find (con compresión de caminos) y Union (por rangos).]